

Flujos por módulo

GamerStore ERP — Cómo viaja cada solicitud del panel admin, de la pantalla a la base de datos y de vuelta

El viaje de CUALQUIER petición (ida y vuelta)

IDA: Pantalla admin → endpoints.js → client.js (+token) → Controller (/api) → Service (reglas) → Repository → Base de datos

VUELTA: Base de datos → Repository → Service → Mapper (Entidad→DTO) → Controller (JSON) → client.js → setState → la pantalla se actualiza

Qué es cada pieza

- **Pantalla** — archivo `frontend/src/pages/admin/...` y la función (handler) que dispara el botón/input.
- **endpoints.js** — `frontend/src/api/endpoints.js`: define la función que arma la URL.
- **client.js** — `frontend/src/api/client.js`: hace el fetch, agrega `Authorization: Bearer` y maneja errores/refresh.
- **Controller → Service → Repository → Mapper/DTO** — `src/main/java/com/gamerstore/app/...`

💡 **Idea clave:** memoriza el viaje de arriba. Cada botón del panel es una variación de ese mismo recorrido; solo cambian el archivo del Controller, el método del Service y la consulta del Repository.



Sesión (transversal a todo el ERP)

► Iniciar sesión (“Ingresar al ERP”)

- 1 **Login.jsx** → `submit()` → **AuthContext.login()** → **AuthAPI.login()** → `POST /api/auth/login` (sin token todavía).
- 2 **AuthController.login()** → **AuthenticationManager** valida con **CustomUserDetailsService** (tabla `usuario`) + BCrypt.
- 3 **JwtService.generarAccess()** firma el access (5 min) y **RefreshTokenService.crear()** guarda el refresh en tabla `refresh_token`.
- 4 Devuelve **LoginResponse** {accessToken, refreshToken, username, nombre, rol}.
- 5 **Front:** guarda tokens + usuario en `localStorage` y entra a `/admin`. **UI:** aparece el contador de sesión.

► Cualquier petición autenticada

- 1 **client.js** agrega `Authorization: Bearer <gs_token>` a la petición.
- 2 **JwtAuthenticationFilter** valida firma + expiración; **SecurityConfig** exige rol ADMIN en `/api/admin/**`.
- 3 Token válido → sigue al Controller. Token inválido → **401**.

► Token vencido → refresco silencioso (automático)

- 1 Una petición devuelve **401** (el access venció a los 5 min).
- 2 **client.js** llama **una vez** a `POST /api/auth/refresh` con el refresh.
- 3 **RefreshTokenService.rotar()** valida el refresh en BD, lo revoca y crea uno nuevo (rotación). Devuelve tokens nuevos.
- 4 **client.js** guarda los nuevos y **reintenta** la petición original sin que el usuario lo note. Si el refresh ya no vale → a `/admin/login`.

► Recargar la página (F5) / Cerrar sesión

- 1 **Restaurar: AuthContext** (al montar) llama `GET /api/auth/me` → **AuthController.me()** devuelve el usuario → la sesión sigue.
- 2 **Logout: Sidebar.jsx** → **AuthAPI.logout()** → `POST /api/auth/logout` → **RefreshTokenService.revocar()** marca el refresh como revocado; el front limpia `localStorage`.

Contador de sesión: SessionTimer.jsx NO hace peticiones; solo lee el campo exp del token y muestra la cuenta atrás; se reinicia solo al refrescarse.

1 Dashboard (panel de inicio del ERP)

► Cargar el dashboard (al entrar a /admin)

- 1 **Dashboard.jsx** → **AdminAPI.dashboard()** (`GET /api/admin/dashboard`) + **AdminAPI.productos()**.
- 2 **AdminDashboardController.dashboard()** junta: `count()` de cada repositorio (productos/categorías/clientes/pedidos); **PedidoRepository.sumTotal()** (ventas); **ProductoRepository.findByStockLessThanEqual...()** (stock bajo); **PedidoRepository.topProductos()** (agrupa `pedido_item` por producto y suma cantidades).
- 3 Devuelve **DashboardDTO** {totales, totalVentas, stockBajo[], topProductos[]}
- 4 **UI**: tarjetas KPI, gráfico de barras (productos por categoría), dona de estado de stock, lista de stock bajo y panel “Más vendidos” (Recharts).

2 Productos

► Listar productos

AdminProductos.jsx · `cargar()` → **AdminAPI.productos()** → `GET /api/admin/productos` → **AdminProductoController.listar()** → **ProductoService.todos()** → **ProductoRepository.findAll()** → tabla `producto` → **ProductoMapper.toDTO()** → **UI**: tabla.

Buscar / ordenar / **paginar** en la tabla NO llama al backend: lo hace el hook **useTableControls.js** en el navegador sobre la lista ya cargada.

► Subir imagen (input de archivo)

- 1 **AdminProductos.jsx** → `subirImagen()` → **AdminAPI.subirImagen(formData)** → `POST /api/admin/uploads` (multipart + token).
- 2 **UploadController.subir()** valida tipo/tamaño (≤5 MB), guarda con nombre UUID en `uploads/productos/` y devuelve **UploadResponse** `{url:"/images/productos/xxx.jpg"}`.
- 3 **Front**: guarda esa **ruta** en `form.imagen` + vista previa. El archivo NO va a la BD, solo la ruta.

► Crear producto ("Guardar")

- 1 **guardar()** arma `{nombre, descripcion, precio, stock, imagen, categoriaId}` → **AdminAPI.crearProducto()** → `POST /api/admin/productos`.
- 2 **AdminProductoController.crear()** (`@Valid`) → **ProductoService.crear()**.
- 3 **Regla**: **ProductoRepository.existsByNombreIgnoreCase()** → si existe, **409** "Ya existe un producto con ese nombre".
- 4 Si no → asocia categoría + **ProductoRepository.save()** → tabla `producto` → **Mapper** → **DTO**.
- 5 **UI**: toast "Producto creado" + tabla refrescada (o toast rojo si hubo duplicado).

► Editar / Eliminar producto

- 1 **Editar**: **AdminAPI.actualizarProducto(id, ...)** → `PUT /api/admin/productos/{id}` → **ProductoService.actualizar()** [valida con `existsByNombreIgnoreCaseAndIdNot`] → `save()`.
- 2 **Eliminar**: confirmación (**useConfirm**) → **AdminAPI.eliminarProducto(id)** → `DELETE .../{id}` → **ProductoService.eliminar()** → `deleteById()` → toast.

3 Categorías

► Listar / Crear / Editar / Eliminar

- 1 **Listar:** `AdminAPI.categorias()` → `GET /api/admin/categorias` → `CategoriaService.listar()` → `CategoriaRepository.findAll()` → tabla `categoria`.
- 2 **Crear:** `AdminAPI.crearCategoria({nombre})` → `POST` → `CategoriaService.crear()`.
- 3 **Regla:** `existsByNombreIgnoreCase()` → si existe, "La categoría ya existe". Si no → `save()`.
- 4 **Editar:** `PUT .../{id}` → `actualizar()` [valida con `...AndIdNot`].
- 5 **Eliminar:** `DELETE .../{id}` → `eliminar()`: si `ProductoRepository.existsByCategoriald()` es true (tiene productos), lo BLOQUEA; si no → `deleteById()`.

4 Clientes

► Listar

`AdminAPI.clientes()` → `GET /api/admin/clientes` → `ClienteService.listar()` → `ClienteRepository.findAllOrderByApellidosAscNombresAsc()` → `ClienteMapper.toDTO()` → UI: tabla.

► Buscar por DNI (“Buscar” — datos reales de RENIEC)

- 1 `AdminClientes.jsx` → `buscarDni()` (valida 8 dígitos) → `AdminAPI.buscarDni(dni)` → `GET /api/admin/clientes/reniec/{dni}`.
- 2 `AdminClienteController.reniec()` → `ReniecService.consultarDni()` llama a `apiperu.dev` (RestClient con timeouts, con el token).
- 3 *Best-effort*: si la API falla → vacío → el endpoint responde **404**. Si responde → `ReniecPersona` {nombres, apellidos, nombreCompleto}.
- 4 UI: autocompleta nombres/apellidos + toast “Datos obtenidos de RENIEC”.

► Crear / Editar / Eliminar

- 1 Crear: `AdminAPI.crearCliente()` → `POST` → `ClienteService.crear()`.
- 2 Reglas: `existsByDni()` → “El DNI ya está registrado”; `existsByEmailIgnoreCase()` → “Ese email ya está registrado”. Si no → `save()` en tabla `cliente`.
- 3 Editar: `PUT .../{id}` → `actualizar()` [valida con `...AndIdNot`; el DNI se muestra bloqueado].
- 4 Eliminar: `DELETE .../{id}` → `deleteById()`.

5 Pedidos

► Listar pedidos + datos del formulario

- 1 **AdminPedidos.jsx** (useEffect) → 3 peticiones: **AdminAPI.pedidos()** (GET /api/admin/pedidos), **clientes()** y **productos()** (para los selectores).
- 2 **PedidoService.todos()** → **PedidoRepository.findAllByOrderByFechaDesc()** → **PedidoMapper.toDTO()** (incluye cliente, ítems y total).
- 3 **UI**: tabla con Código, Cliente, Fecha, Ítems, Total, Estado.

► Registrar un pedido (“Registrar”)

- 1 Se agregan ítems con **agregarItem()/quitarItem()** (en memoria, total calculado en el front). Al enviar → **guardar()** → {clienteId, metodoPago, items:[{productid, cantidad}]} → **AdminAPI.crearPedido()** → POST /api/admin/pedidos .
- 2 **PedidoService.crear()**: busca el cliente; por cada ítem busca el producto (**ProductoRepository.findById**), crea un **PedidoItem** con el precio actual y **suma el subtotal** al total.
- 3 **PedidoRepository.save()** guarda el **pedido** y sus **pedido_item** juntos (cascade).
- 4 **UI**: toast “Pedido registrado” + tabla refrescada.

► Editar estado / Eliminar

- 1 **Editar estado**: **AdminAPI.actualizarPedido(id, {estado, metodoPago})** → PUT .../{id} → **PedidoService.actualizar()** → **save()** → el badge cambia de color.
- 2 **Eliminar**: **DELETE .../{id}** → **deleteById()** (borra el pedido y sus ítems por cascade).

► Descargar reporte PDF (botón + filtros fecha/estado)

- 1 **descargarPDF()** arma la URL con filtros (**AdminAPI.reportePedidosUrl()**) → **downloadBlob()** hace GET /api/admin/pedidos/reporte.pdf?... con token y recibe un blob.
- 2 **AdminPedidoController.reporte()** → **PedidoService.reporte()** (filtra por fecha/estado) → **PedidoReporteService.generar()** arma el PDF con **OpenPDF** (encabezado, tabla, total) con Content-Disposition: attachment.
- 3 **Front**: **downloadBlob** crea un enlace temporal y descarga **reporte-pedidos.pdf**.

6 Usuarios del sistema

► Listar / Crear / Editar / Eliminar

- 1 **Listar:** `AdminAPI.usuarios()` → `GET /api/admin/usuarios` → `UsuarioService.listar()` → `UsuarioRepository.findAllOrderByUsernameAsc()` → `UsuarioMapper.toDTO()` (sin la contraseña).
- 2 **Crear:** `POST` → `UsuarioService.crear()`.
- 3 **Reglas:** `existsByUsername()` → “Ese usuario ya existe”; `existsByEmail()` → “Ese email ya está registrado”. La contraseña se **hashea con BCrypt** antes de `save()`.
- 4 **Editar:** `PUT .../{id}` [valida con `...AndIdNot`; si la clave viene vacía, no la cambia].
- 5 **Eliminar:** si es el **último ADMIN** (`countByRol(ADMIN) <= 1`) lo BLOQUEA; si no → `deleteById()`.



Cómo viaja un ERROR/duplicado hasta el toast

► Ejemplo: crear una categoría repetida

- 1 El **Service** lanza una excepción con mensaje (ej. `ResponseStatusException(409, "La categoría ya existe")`).
- 2 **GlobalExceptionHandler** (`@RestControllerAdvice`) la convierte en `{ "error": "La categoría ya existe" }` con el código HTTP correcto.
- 3 **client.js** ve que no es 2xx, lee ese error y lanza `Error(mensaje)`.
- 4 La pantalla, en el catch del `guardar()`, hace `toast.error(mensaje)` (**Toast.jsx**).
- 5 **UI**: toast rojo con el mensaje exacto; el registro NO se crea.

Códigos que devuelve el backend: 400 datos inválidos · 401 no autenticado / credenciales malas · 404 no encontrado · 409 duplicado o en uso · 413 imagen muy grande.